

Agora — The Deliberation Engine

A technical and theoretical description, for researchers and AI systems

Agora — The Deliberation Engine

- 0. How to read this document
- 1. The problem
- 2. Design commitments
- 3. Session architecture
- 4. The deliberation cycle
- 5. The measurement subsystem
- 6. The improvement economy
- 7. AI in the loop
- 8. Outcome determination
- 9. Where the maths lives
- 10. Threat model and known failure modes
- 11. Testable predictions
- Appendix A — Constant reference
- Appendix B — Correspondence to *On Deliberation*
- Appendix C — Further reading in this repository

Agora — The Deliberation Engine

A technical and theoretical description, written for researchers and for AI systems reasoning about the codebase.

Agora is a 45-minute classroom deliberation, staged as a rescue mission through time. Beneath the game is a working deliberation engine: a rating scale, a consensus estimator, a cross-camp bridging score, an attention allocator, and an incentive economy for mutual improvement.

This document explains **what the engine does, how each mechanism works, and why it was built that way** — including the places where the implementation deliberately departs from its own theoretical source, and the failure modes it is known not to defend against.

- **Implementation:** `apps/agora` (Mithril + Vite, Hebrew-first RTL), `functions/src/agora` (Cloud Functions, me-west1), `packages/shared-types/src/models/agora` (schemas and *all* shared maths).
 - **Theoretical source:** Tal Yaron, *On Deliberation: A Cultural, Methodological, and Technological Framework for Collective Decisions*. Chapter references below are to that text.
 - **Sibling systems:** the same consensus formula runs in Freedi's main app, MassConsensus, and the Sign app (`packages/shared-types/src/utis/consensusCalculation.ts`). Agora adds a finite-population correction that the others may pass or omit.
-

0. How to read this document

Sections 1–2 give the problem and the normative commitments. Sections 3–8 describe mechanisms in execution order. Sections 9–11 are for anyone extending, auditing, or measuring the system. Appendix A is a complete constant reference; Appendix C is a line-by-line correspondence to the theory.

Every numeric claim in this document is traceable to a named constant or function. Where a value is a judgment call awaiting calibration, it is marked **[uncalibrated]**.

1. The problem

1.1 Deliberation vs. aggregation

The framework Agora implements — *grounded selection* — distinguishes two things that voting systems fuse:

1. **Grounding.** Building a shared, corroborated picture of how the relevant part of the world works, good enough to *forecast what a candidate solution would do*.
2. **Selection.** Filtering an open field of proposals down to those that command sufficient support, using a measure honest about its own uncertainty.

A vote does neither. It samples preferences over a fixed menu, at a single moment, with no record of intensity, no account of what the voters believed, and no mechanism by which a proposal can improve between being written and being judged.

The theory's success criterion is not a majority but **expanding agreement**: a level of net support high enough (the book proposes ~80% as a working threshold) that participants whose preference did not prevail are positioned to live with the result rather than resist it. Where no proposal clears that bar, the process is expected to produce **honest disagreement** — a precise account of *where* the parties differ — which the framework treats as a genuine result, not a failure.

1.2 Why a classroom

Chapter 7 of the source reasons almost entirely about populations too large for any participant to survey: thousands of proposals, sampled evaluation, statistical humility that never fully retires. A classroom inverts three of those premises at once, and the engine exploits all three:

LARGE-POPULATION CASE	CLASSROOM CASE	ENGINE CONSEQUENCE
Stakeholder set unbounded and contested	Known, finite, fully present	Finite-population correction (§5.3)
Nobody can evaluate everything	Near-census coverage is achievable	Rating budget sized to the class; <code>consensusCeiling</code> normalisation (§5.4)
Opinion clusters must be <i>inferred</i> from the rating matrix	Camps can be <i>declared</i> before anyone writes	Bridging computable from the first rating (§5.5)

The cost of the inversion is that everything Chapter 9 says about *constituting* a deliberation — breaking the “how do we decide how to decide” recursion, drawing the stakeholder boundary, framing the question across perspectives — is out of scope. The class is the stakeholder set; the question arrives in the topic package. See §10.4.

2. Design commitments

These are the load-bearing normative decisions. Each is stated as a commitment, the mechanism that enforces it, and the specific failure it exists to prevent. Where a commitment is enforced by a *type* or a *state machine* rather than by convention, that is deliberate: the project rule is “if something must never happen, express it in a type or a security rule, not a comment.”

D1 — Judgment is graded, not binary

Mechanism. Five levels: `AGORA_RATING_LEVELS = [-1, -0.5, 0, +0.5, +1]`. **Why.** A yes/no record cannot distinguish a proposal that most people mildly support and a few oppose absolutely from one everybody finds tolerable. Direction *and* intensity are needed for the consensus estimator (§5.2) to penalise strong minority objection. **Prevents.** Majority rule's structural blindness to intensity of opposition.

D2 — Every student writes before seeing anyone else's proposal

Mechanism. The deliberation cycle begins on the `mine` step; `screenForStep('mine') === 'my'`, and classmates' proposals are not reachable from that screen (`lib/flows/deliberationFlow.ts`). **Why.** Early visible answers collapse the diversity of independent estimates — the mechanism behind information cascades and groupthink. The remedy is to collect independent judgments *before* exposure. **Prevents.** Cascade convergence on whichever proposal happened to be written first.

D3 — Attention is allocated, not left to timing

Mechanism. `rankStalls / orderSquare` in `apps/agora/src/lib/squareOrder.ts`. **Why.** Under any natural default, proposals submitted early are seen by more people simply because they have existed longer, accumulate more evaluations, and rise on timing rather than merit. **Prevents.** “Who gets heard” being decided by submission order.

D4 — Reaching across camps is worth more than being popular at home

Mechanism. `calcBridgingScore` weights cross-camp support 0.65 against same-camp 0.35. Own-camp support alone caps the score at 35, below both credit thresholds (70, 80), so **no bridging tier is geometrically reachable without cross-camp support**. **Why.** The object grounded selection searches for is a proposal that draws support *across* the opinion clusters rather than rallying one against the others. **Prevents.** A camp-captured proposal winning by turnout.

D5 — Improving someone else’s proposal is paid, and loudly

Mechanism. The improvement economy (§6): a thanked idea pays the helper, weaving pays the author *per distinct helper*, and the improvement celebration is the game’s loudest audiovisual moment. **Why.** The theory argues that a proposal under heavy criticism has already improved the deliberation, and that groups exposed to dissent reason more divergently — but offers this as something a facilitator *says*. Agora makes it something the system *pays for*. **Prevents.** Criticism being experienced as attack, and the resulting withdrawal (exit rather than voice).

D6 — Silence is not neutrality

Mechanism. In `calcBridgingScore`, a camp with `n === 0` contributes 0, not `warmth(0) = 0.5`. In `agoraClassSupport` and `calcAgoraClassConsensus`, absence returns `undefined`, never 0. **Why.** A proposal nobody read must not score like one the class considered and shrugged at. On a $-1 \dots +1$ scale, 0 means “unanimously, maximally against” — the opposite of “not yet spoken”. **Prevents.** Unread proposals inheriting a default midpoint and outranking examined ones.

D7 — Disagreement is not launderable by counting more of it

Mechanism. The finite-population correction (§5.3) removes *sampling* error only. `polarization (1 - calcLikeMindedness)` deliberately takes no population size. **Why.** At a census the mean is observed rather than estimated, so the confidence penalty must vanish — but a class that genuinely splits 8-for/7-against must still score at or below zero however completely it is counted. **Prevents.** A divided class manufacturing consensus by achieving full turnout.

D8 — The AI never writes for a student

Mechanism. Invariant I6 (`docs/chat-guide-rules.md`). The guide may prompt, quote, react, and route. It never drafts, rewrites, or “improves wording”. AI critique exists **only** as in-character reviews of a proposal the student wrote. **Why.** Stated by the project owner as: “*otherwise they will not think.*” Note this is **stricter than the theoretical source**, which endorses real-time LLM rephrasing of barbed messages (§7.3). **Prevents.** Students evaluating a machine’s words rather than each other’s.

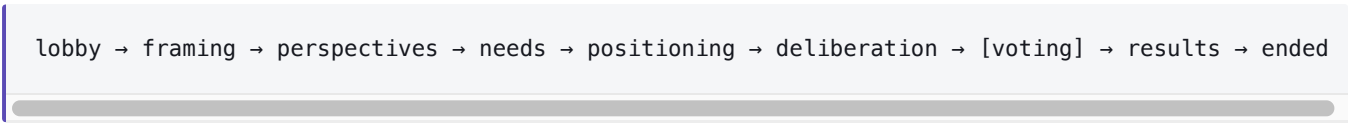
D9 — Every number a student sees is server-written or computed by shared code

Mechanism. All scoring maths lives in `packages/shared-types` , imported by both the client and the Cloud Functions. `agoraScores` documents are written **only** by the admin SDK; client writes are denied by security rules. **Why.** The client and the trigger once counted the class differently, so the projector and the phones disagreed about the same proposal. A score that disagrees with itself in front of a class discredits every other number on the screen. **Prevents.** Divergence between the shared display and individual devices.

3. Session architecture

3.1 Stages

`AgoraStage` (`agoraEnums.ts`), advanced by the teacher:



`valueIdentification` remains in the enum but is **removed from the flow** — a second AI-graded writing task immediately before proposal writing was judged too much cognitive load for 45 minutes. The grading infrastructure (`fn_agoraGradeValueIdentification`) is retained but unused.

`voting` is optional; a teacher may advance `deliberation -> results` and never hold an election.

STAGE	STUDENT ACTION	DELIBERATIVE FUNCTION
lobby	Join by 5-digit code; anonymous marker appears	Anonymity established before any content
framing	Watch period explainer	Grounding — the shared picture needed to forecast
perspectives	Two characters argue, both sympathetic	Positions collide, visibly
needs	Characters restate needs; needs board persists	Position → need translation
positioning	Place self 0–100 between the two characters	Declares camp; makes bridging measurable
deliberation	5 self-paced laps (§4)	Propose, evaluate, improve
voting	Elect among top-N by consensus	Optional closure
results	Forecast, verdict, debrief	Closing on expanding agreement or honest disagreement

3.2 Camp derivation

```
campPosition ≤ 40      → AgoraCamp.left
campPosition ≥ 60      → AgoraCamp.right
40 < campPosition < 60 → AgoraCamp.center
```

Camps are **declared, not inferred**. At $n \approx 30$ there is no rating matrix rich enough to cluster reliably, and a declared camp makes the cross-camp weight computable from the very first rating.

The trade-off is that the bridging score is only as meaningful as the honesty of positioning. The sole defence is framing: “*the game rewards you for building bridges **from** where you actually stand, not for standing in the middle.*” This is a known soft spot — see §10.2.

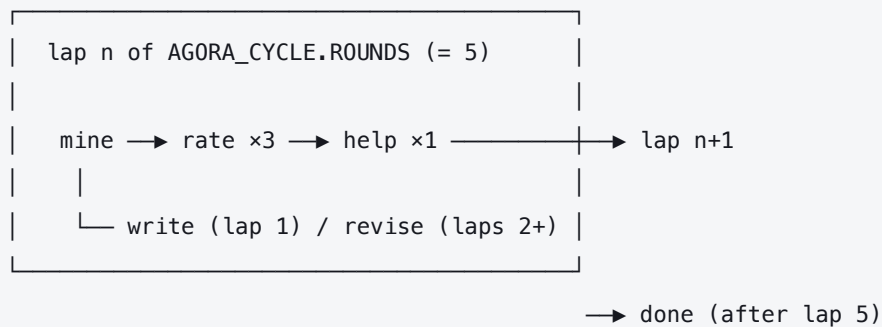
Civic mode (`AgoraSessionMode.civic`, used by the Odyssey app) skips positioning entirely and derives each participant’s camp from stances they already took elsewhere. Same engine, different camp source.

3.3 Anonymity

Proposals are identified by number, never by name. Individual ratings are **never** shown to proposal owners — only aggregates. Rating *times* stream to everyone (needed by the improvement loop, §6.4); rating *values* never leave the server in identified form.

4. The deliberation cycle

The core loop. Each student runs it independently and self-paced; there are no teacher-synchronised phases inside the deliberation stage.



- **mine** — write the first draft (lap 1) or revise it in light of feedback (laps 2+). Scaffold phrasing: “*a solution I propose, that we could do together*” — the proposal is owned and addressed to joint action.
- **rate** — `AGORA_CYCLE.RATINGS_PER_ROUND = 3` classmates’ proposals, own excluded, served in the order §5.7 defines.
- **help** — one improvement suggestion for a classmate, or skip. Skipping is free and unremarked (guilt here would push students toward giving +1 ratings just to avoid the follow-up prompt, which would corrupt the rating signal).

The rules of the lap live in `lib/flows/deliberationFlow.ts` as a pure state machine with injected dependencies — no Mithril, no Firestore — so lap turnover, screen routing, and termination are unit tested in node. It was extracted from a 2,899-line view where the only part of the deliberation with real rules could not be tested without a browser.

The whole square carries a fuse: `AGORA_CYCLE.DELIBERATION_TOTAL_MS = 20 min`. The teacher opens the square and decides when it closes.

5. The measurement subsystem

5.1 Rating storage: a histogram, not a running sum

Each proposal's score document holds, per camp, `{sum, n, positiveN, studentDist}` where `studentDist` is a 5-tuple of counts indexed by `AGORA_RATING_LEVELS`.

Two reasons the histogram exists rather than a running `sumSquares`:

1. **Exact moments.** The levels (1, 0.5, 0.25 as squares) are all exactly representable in binary floating point, so `distMoments` computes `n`, Σe , Σe^2 with no accumulation drift.
2. **Auditable repair.** Negative counts are the fingerprint of a double-applied delta and clamp to zero. A bare `sumSquares` offers no such repair, because nothing about a corrupted one looks wrong.

`agoraRatingBucket` snaps arbitrary values to the nearest level. This is defensive by necessity: the security rules only require the evaluation to be a number, so a buggy or hostile client could write `0.37` or `5`. Snapping bounds the error at ± 0.25 and preserves the invariant the whole design rests on: $\Sigma \text{dist} === n$.

Aggregates are recounted from the evaluations on every rating, not nudged by a delta. The delta path could not survive a rater whose camp arrived late: a student who rated *before* positioning was skipped in the bridging half, and skipped was permanent, because the subsequent edit carried `n = 0`. Recounting is self-healing — the moment a camp is known, every rating that student ever left is in the right column — and at classroom scale one query per rating over one proposal's evaluations is cheap.

5.2 Consensus `C_p`

The shared Freedi estimator (`packages/shared-types/src/utils/consensusCalculation.ts`):

```
C_p    = μ_p - t(α, n+k-1) · SEM*_p

μ_p    = Σe_i / n
δ*_p   = √( Σe_i^2 / (n + k - 1) )
SEM*_p = δ*_p / √(n + k) · fpc(n, N)

k = 2      (BAYESIAN_PRIOR_K - phantom prior votes of 0)
α = 0.05   (one-sided; t-table with linear interpolation, z = 1.645 for df > 120)
```

Two properties do the deliberative work:

Confidence is bought with evaluations and nothing else. The `k = 2` phantom neutral votes mean the system’s working assumption in the absence of data is a *neutral* community, not a supportive one. Three enthusiasts score as promising-but-unproven; the phantoms fade to insignificance as real evaluations accumulate.

Strong opposition is not noise. A wide spread of opinion inflates Σe_i^2 , which inflates `SEM*`, which drags `C_p` down. A proposal 70% of the class loves and 30% fiercely opposes scores *below* one everybody supports moderately. This is the theory’s harm-minimisation standard written into the arithmetic rather than asserted alongside it.

`C_p` is bounded so the result stays in $[-1, 1]$: `boundedPenalty = min(penalty,  $\mu + 1$ )`.

5.3 The finite-population correction

This is Agora’s addition to the shared engine, not an implementation of the source text.

```
fpc(n, N) =  $\sqrt{(1 - \text{clamp}(n/N, 0, 1))}$ 
```

`SEM*` asks “how far might the true opinion of the *world* be from this sample?” — an infinite-superpopulation question. When the stakeholders are a known finite set, that is the wrong question. Once all of them have spoken the mean is **observed, not estimated**, and the sampling penalty must be exactly zero.

COVERAGE	FPC	EFFECT
<code>n = N</code> (census)	0	<code>C_p</code> collapses to μ . A class of six unanimously in favour scores 1, not 0.33.
<code>n << N</code>	$\rightarrow 1$	Recovers the uncorrected formula exactly; a thin self-selected poll stays as humble as before.
<code>n > N</code>	clamped to census	The generous direction, and a data-quality signal that <code>N</code> is wrong.

Integrity constraint. Understating `N` inflates consensus — with 50 respondents, declaring `N = 50` instead of `N = 500` moves `C_p` from 0.420 to 0.600. `N` is therefore treated as *a published claim about who has standing in the decision, not a private tuning dial*: any surface showing an FPC-corrected score is required to show `n` and `N` alongside it.

Who is in `N`. Two different questions get two different pools, and conflating them was a real bug:

- **Bridging** divides by *positioned* students per camp — a student whose side nobody knows cannot support a claim about reaching across camps.
- **Consensus** divides by the *whole class*, with unpositioned students filed under centre, because their ratings still count. Leaving them out made a proposal read “1 of 0 rated”.
- The **author’s own seat is removed** from the eligible pool (`eligiblePoolFor`), because the square never serves anyone their own text; counting it would leave a fully-participating class permanently one rating short of a census.
- A pool smaller than the raters who actually turned up is treated as stale data, not a census, and falls back to `max(eligible, n)`.

5.4 Normalisation against the achievable ceiling

```
consensusCeiling(n, N) = calcAgreement(n, n, n, N)    // n ratings all at +1
normalized              = clamp01( C_p / ceiling )
```

A student's rating budget is fixed ($\text{ROUNDS} \times \text{RATINGS_PER_ROUND} = 15$). With one proposal per student, coverage $\approx 15 / (\text{class} - 1)$ — a full census up to about sixteen students, then falling away. At identical class sentiment, raw `C_p` therefore drops sharply as the class grows, and the success threshold would be easy at six students and arithmetically impossible at forty.

Use `normalized` to compare across classes; use raw `consensus` within one class. The class-score threshold (§8) judges `normalized`.

5.5 Bridging

```
bridging = 100 × ( 0.35 · S_own + 0.65 · S_other · conf )

S_c      = warmth(mean rating from camp c), and 0 if that camp has n = 0
warmth   = (clamp(m, -1, 1) + 1) / 2          -1 → 0 · 0 → 0.5 · +1 → 1
conf     = min(1, n_other / max(1, min(3, crossCampPool)))
```

Centre-camp raters count toward **both** wings at `CENTER_CAMP_WEIGHT = 0.5`. A centre-camp author is treated symmetrically: both wings are “other”, and same-camp support comes from the centre itself.

Why `warmth` maps rather than clips. This function was `clamp01(mean)`, which floored every negative mean at 0 and collapsed the entire *against* half of the scale onto one value. The consequence was not cosmetic: a classmate moving from “strongly against” to “neutral” — a real change of mind, and the biggest one a revision can win — moved the bridging score by *exactly nothing*. The improvement loop then reported “it hasn’t moved yet” to an author whose revision had worked, in precisely the situation where a proposal most needs rewriting. The score was blind on the half of the scale the game is about. Thresholds were re-pointed by the same arithmetic so the sentiment a credit costs is unchanged (both camps at mean +0.6 scored 60 under the old term and 80 under this one).

Why `conf` has a variable denominator. A fixed `/3` makes a high bridging score arithmetically impossible in a small class: with four students there are at most one or two cross-camp raters, so confidence caps at `1/3` forever and no credit is reachable. `crossCampPoolFor` asks the honest question — *of the cross-camp students that exist, how many support this?* — without ever inflating past `MIN_CROSS_RATERS` in a full class.

The ladder. Two rungs, paid cumulatively and monotonically:

TIER	THRESHOLD	PAYOUT	MEANING
1	70	+5	“You reached across”
2	80	+10 (15 cumulative)	Full bridge

A single cliff at the top meant most authors never felt the mechanic at all. `bridgingTierAwarded` is monotonic: a later dip never claws a tier back, so an author is never punished for a proposal that moved. A

class that is merely indifferent — every camp at mean 0 — reads **50**, deliberately below tier 1: *shrugging is not bridging*.

5.6 Polarization

```
polarization = 1 - calcLikeMindedness( $\Sigma e$ ,  $\Sigma e^2$ , n)
calcLikeMindedness = clamp01(1 - SEM*)           // no population size, by design
```

Applying the FPC here would make a perfectly split group that everyone voted in report 1.0 — the exact opposite of the truth. “How divided are they” must stay independent of “how many of them we heard from”.

Known limitation, inherited from the shared engine: `SEM*` is built from the sum of squares, which discards sign, so six votes of +1 and a 3–3 split of ± 1 produce the identical value. No single scalar here currently detects polarisation; a divided class is visible only in the distribution.

5.7 Attention allocation

Two orderings, for two different questions.

orderSquare — the row a student browses. Freshest *writing* first, where “writing” means the author’s own two clocks: `createdAt` (they posted) or `agoraScores.lastEditAt` (they rewrote). Rating is deliberately not a clock, and neither is the statement’s `lastUpdate`: the evaluation pipeline bumps that on every aggregate write, so ordering on it reshuffled the whole square each time anybody anywhere pressed a face — rows moved under a reading finger for a reason the reader could not see. Ties break on a deterministic per-student hash.

rankStalls — the order a lap deals help targets. Three rules, in priority order, each about spreading attention rather than ranking work:

1. Proposals I have **not** helped come first — my second idea on the same text is worth less to the class than my first idea on a text nobody has read.
2. Then the ones with the **fewest ideas already waiting**, so help lands where there is none rather than piling onto whoever got noticed first.
3. Then the **per-student hash**, so two equally-neglected proposals do not send the entire class to the same one.

Computed once per lap and held steady; `mergeLateArrivals` appends a classmate who posts mid-lap to the *end* of the row rather than re-sorting them in, and the next lap re-ranks everything anyway.

What is deliberately absent. The source text’s larger allocator also boosts newly-submitted proposals, leans toward near-threshold ones, adds bandit-style randomness, and retires proposals once their standing is reliable. None is implemented: at ~30 proposals against a 15-rating budget the field is near-census, so the exploration/exploitation problem those mechanisms solve does not arise.

6. The improvement economy

The game's core loop is not "write and be judged". It is **write** → **get helped** → **say thank you** → **improve** → **be re-judged**. Full specification: `docs/feedback-cycle.md`.

6.1 The cycle

```
A writes proposal
B rates it (5-level)
B opens the conversation and sends an improvement      [status: open]
  |
  └─ A says 🙏 thank you → B: celebration, +1 point
  └─ A passes           → B: quiet toast, no cost
A edits the text and saves
  → B sees a word-level diff, and is asked to weigh the new version
B re-reads and re-rates
  → B: 🗨 "Round Trip" – the circle, named at the moment it closed
  → A: "N classmates re-rated · support 50 → 62", in B's thread
```

Two things happen *to* a proposal rather than being said about it, and both are written into the thread as system lines by Cloud Functions:

- 🖋 **the author changed the text** — with a word-level LCS diff (`lib/textDiff.ts` ; character diffs on Hebrew were unreadable). Written by `onAgoraProposalWritten`, the only place holding both versions — a client-written record of “what it used to say” would be one anyone could forge. It carries **no thread uid**: an edit belongs to every conversation about that proposal.
- 🏆 **what a thank-you paid** — written by `agoraResolveSuggestion` beside the idea that earned it, so the number can never drift from the transaction that produced it.

Neither is a message. `isSuggestionKind` is an allow-list, so system lines never occupy an open-idea slot, never ask the author for a decision, and never fire a “somebody is talking to you” toast.

6.2 Point schedule and its incentive logic

ACTION	POINTS	CAP	RATIONALE
First proposal	+3	once	The steepest step of the funnel, and it used to pay nothing. Deliberately below a landed idea's total.
Rating a proposal	+0.5	15 ratings	Pays for the commons the whole game runs on (bridging confidence, coverage). Value-blind — identical for “strongly against” and “strongly for”, so there is no incentive to rate in any direction. First rating of a given proposal only; the deterministic evaluation id dedupes re-rating.
Being thanked (helper)	+1	one per proposal per helper	Bounded by construction: only a helper's <i>first</i> message on a proposal is suggestion-kind.
Being declined (helper)	0	—	A penalty was regressive: the floor at 0 exempted the spammer with an empty balance and taxed only the productive helper who had points to lose. Spam is bounded structurally instead (§6.3).
Weaving a helper's idea (author)	+1 per distinct helper	3 per proposal	Integrating many voices beats trading rounds with one buddy — the incentive is bridging-shaped by construction.
Revising after feedback (author)	+1	3	Writing, rating and helping all earned; revision did not, and students read that asymmetry as “revision doesn't count”.
Bridging tier 1 / tier 2	+5 / +10	cumulative, monotonic	\$5.5

(Legacy: `SUGGESTION_ACCEPTED = 1` and `SUGGESTION_IMPLEMENTED = 2` belong to the retired accept → weave chain. The thank-you now carries the whole helping side of the economy.)

6.3 Anti-gaming

GUARD	VALUE	WHAT IT BINDS
<code>MAX_OPEN_SUGGESTIONS_PER_HELPER</code>	2	The real spam guard — it binds on the spammer regardless of balance, unlike a points penalty. Resolved ideas free the slot.
<code>MAX_WOVEN_AWARDS_PER_HELPER_PER_PROPOSAL</code>	2	Bounds the collusion loop (A and B trading rounds forever) with no surveillance. Past the cap the celebration still fires — recognition is decoupled from currency — but the points do not.
<code>MIN_REVISION_DELTA_WORDS</code>	3	Below this a save is cosmetic: still saved, still shown, just not an event.
<code>REVISION_DEBOUNCE_MS</code>	3 min	Five quick polishing saves are one revision, not five. A farmer clicking save gets nothing to farm.
Revision feedback gate	—	A revision credit requires new feedback since the last credit: <code>studentRatingsAtCredit</code> must have moved, or a thank-you must have landed. The same ratings can never pay for two revisions. The credit rewards listening, not saving.
<code>EDIT_HISTORY_MAX</code>	10	Past the cap the first entry (where the journey began) and the newest 9 survive. A pathological save loop cannot grow the score doc without bound.

6.4 Derived, not stored

Three signals in the improvement thread are recomputed from live state on every render (`lib/improvementSignals.ts`) rather than fired and recorded. This is why none can double-fire, go stale, or need a watermark — and why the whole feature needed no schema, rules, or Cloud Function change.

- **The re-weigh block** (helper side) — shown while the owner has revised after my idea and I have not weighed the new version. Clears the instant the rating lands, from any surface. It deliberately does **not** claim my idea is in the text: the trigger is an edit that *followed* my idea, and the diff above it is the evidence the student judges for themselves.
- **The credited score line** (owner side) — rendered in the **one** thread whose helper the owner acknowledged most recently before saving, ties broken on uid so every device in the room computes the same helper. Without a single answer, an owner who thanked three classmates before one save would see the same “+12” claimed in three different conversations.

-  **Round Trip** — history rather than state, so it carries the moment it closed and sorts into the conversation by it. **Direction-blind by design:** a round trip that only counted when the score rose *would be paying the helper to vote up* — the exact pressure the re-weight block exists to resist.

Attribution discipline. The helper caused the revision; the class caused the score. The credited line is credited by *placement* — it renders in that helper’s thread — while the numbers stay attributed to the class. Fusing the two into “your score rose because of X” would denominate a classmate’s goodwill in points, and would hand them the blame the next time it fell.

Which number the author is shown. The improvement loop reports the **plain class mean** (`agoraClassSupport`), not the bridging score. Bridging is a composite — weighted, damped by a confidence ramp — so one classmate genuinely changing their mind can move it by less than a point and round away to nothing. Telling an author who revised and won someone over that “it has not moved” is the one failure this loop cannot afford.

Clock discipline. `editClock` reads *only* `agoraScores.lastEditAt`, stamped server-side on real text changes, with no fallback to the statement’s `lastUpdate` — that clock moves when the evaluation pipeline writes aggregates and when any child document is written, including the reader’s *own* suggestion. Trusting it announced “the proposal was revised” to a helper the moment they finished writing. `answerBaseline` *does* fall back, because the two questions fail in opposite directions: “did the owner revise?” wrongly must not invent an edit, while “who answered since I saved?” wrongly must not inflate the count.

7. AI in the loop

Three AI roles, with sharply different permissions.

7.1 In-character reviewers — AI as a stakeholder proxy

A student may show their proposal to either character. The character answers in voice with a verdict, an acceptance score 0–100, and concrete advice keyed to that character’s needs. **That verdict enters the real evaluation pipeline** as `RATERS_PER_CHARACTER = 3` synthetic raters positioned in that character’s camp (`LEFT_CAMP_POSITION = 10`, `RIGHT_CAMP_POSITION = 90`), with uids prefixed `agora-ai--`.

```
agoraScoreToEvaluation(s) = round((clamp(s,0,100)/50 - 1) × 100) / 100
                        e.g. 33 → -0.34
```

Consequences, all deliberate:

- Winning over the *other* side’s character visibly blends the proposal’s colours, exactly as cross-camp classmates would. The mechanic teaches by making the abstraction physical.
- Synthetic raters move `sum / n / positiveN` (which bridging reads) but **never** the student histogram — their values are off-grid and no five-level bucket can hold them, and they are not members of the class whose finite population `N` counts.
- They are excluded from every outcome statistic (`isAgoraAiUid`), otherwise asking both characters would trivially satisfy “rated by both camps”.

- `MAX_ASKS_PER_CHARACTER_PER_ROUND = 5` enforces *improve first, then ask again* — feedback → revise → verify, the improvement loop in miniature. A verdict older than the proposal's latest edit is marked visibly stale and re-asking becomes the primary action.

7.2 The results evaluator

One batched call at `results` scores every proposal for **historical plausibility** against the topic package's weighted rubric, simulates the national health metrics under the leading proposal, and writes a formative class debrief. Deterministic fixtures replace it when `OPENAI_API_KEY` is absent, so emulators, e2e, and CI are reproducible.

The debrief prompt is constrained: grounded in the real deliberation statistics it is given, *never shaming*, and forbidden from declaring victory or defeat — the outcome is computed, not narrated.

7.3 What the AI is forbidden to do

The guide **never drafts, rewrites, or improves student wording** (D8). It may prompt, quote, react, and route. It never injects unprompted messages either: external events surface as badges, counts, or a nudge line on the *currently active* card — never as new bubbles pushed into the log while a student is mid-task. An injected line scrolls the conversation and yanks the current task out from under the student's thumb; the guide waits its turn, and the legitimate turn is the moment the student completes an action.

This is **stricter than the theoretical source**, which endorses real-time LLM rephrasing of barbed messages on evidence that it improves tone and felt understanding without changing content. Agora declines that affordance for a pedagogical reason that does not apply to adult civic deliberation.

8. Outcome determination

8.1 The lead proposal

The crown goes to the class's own consensus, not the bridging score — a proposal leads because the class agrees with it. Where a vote was held **and its winner cleared the teacher's consensus threshold**, the elected proposal outranks the consensus reading and is what the health-metric simulation forecasts.

Votes are read from the `votes` collection rather than from the question's maintained tallies, because a vote cast a moment before the teacher advanced may not have reached the tally yet. A winner that misses the bar is still *named* — the class elected it and the screen must say so — but does not take the crown.

8.2 Class score

```
total = round( 0.45 · consensusTerm
              + 0.25 · min(100, avgPoints)
              + 0.30 · avgPlausibility )

consensusTerm = round(100 × leadConsensus.normalized)    // falls back to maxBridging on legacy c
avgPoints      =  $\Sigma$  points over non-AI participants / class size
avgPlausibility = mean AI plausibility across proposals
```

8.3 Three endings

```
deriveAgoraOutcome(total, threshold, crossRatedProposals, raterCoverage):
```

```
total ≥ 70                                → success
else if crossRatedProposals ≥ 2
    and raterCoverage ≥ 0.5                → honestDisagreement
else                                       → collapse
```

- **success** — a decision at expanding agreement. The Terror never happens.
- **honestDisagreement** — no proposal cleared the bar, *but the class demonstrably deliberated*: at least two proposals were rated by **both** wing camps, and at least half of positioned students rated something. Presented as a dignified ending with a warm AI debrief, not a failure screen.
- **collapse** — below threshold with the evidence of real deliberation missing.

The honest-disagreement gate is the instrumented form of the theory's *confirmed division*: the distinction between **knowing** a population is split and **not having asked enough people to know anything**. A naive reading calls a split bad data; the correct reading is that the measurement succeeded — we know precisely what the class thinks: it is divided.

`crossRatedProposals` and `raterCoverage` are computed from **student evaluations only**, with camps resolved from participant documents and AI uids filtered out.

8.4 A known divergence from the theory

The source's criterion is **~80% net support on one proposal**. Agora's is a composite that also pays for effort and historical plausibility, clearing at 70.

The composite is pedagogically kinder — a class can score well for deliberating well even without convergence — but it is *not* the theory's test, and it lets effort substitute for agreement. The project's own open-questions list proposes the fix: **success should be expanding agreement on a proposal; the composite should become the grade, not the verdict**. Not yet implemented.

9. Where the maths lives

Architectural rule: **shared-types holds every number both the client and the functions use**. A duplicated formula is a formula free to drift, and the projector disagreeing with the phones about the same proposal discredits every other number on the screen.

CONCERN	LOCATION
Consensus, SEM*, FPC, t-table, like-mindedness	<code>packages/shared-</code> <code>types/src/utils/consensusCalculation.ts</code>
Class consensus, histograms, camp tally, eligible pools	<code>packages/shared-</code> <code>types/src/models/agora/agoraConsensus.ts</code>
Bridging, warmth, camp derivation, tiers and payouts	<code>packages/shared-</code> <code>types/src/models/agora/agoraBridging.ts</code>
All tunable constants	<code>packages/shared-</code> <code>types/src/models/agora/agoraConstants.ts</code>
Score document schema (valibot)	<code>packages/shared-</code> <code>types/src/models/agora/agoraScore.ts</code>
Outcome rule	<code>packages/shared-</code> <code>types/src/models/agora/agoraOutcome.ts</code>
Class score, vote counting, AI results	<code>functions/src/agora/classScore.ts</code>
Evaluation trigger (recount, credit, bridging payout)	<code>functions/src/agora/fn_onAgoraEvaluation.ts</code>
Proposal trigger (edit clock, diff line, revision credit)	<code>functions/src/agora/fn_onAgoraProposal.ts</code>
In-character review → synthetic raters	<code>functions/src/agora/fn_agoraCharacterReview.ts</code>
Cycle state machine	<code>apps/agora/src/lib/flows/deliberationFlow.ts</code>
Attention ordering	<code>apps/agora/src/lib/squareOrder.ts</code>
Derived improvement signals	<code>apps/agora/src/lib/improvementSignals.ts</code>

Client boundaries. Components never import Firebase (a widget once owned the app's most important write). Every write goes through `lib/` and returns something the caller can believe — Firestore answers from cache and queues silently, so a write that never lands never rejects either; the only honest signal is a clock (`lib/confirmedWrite.ts`).

Verification. `npm run check-all` (lint, typecheck, tests, build, contrast, type audit) plus five e2e scripts that assert **Firestore state, not pixels** — `e2e-cycle` (the whole improvement loop, asserting points), `e2e-changes` (change chips and seen-state), `e2e-stuck-write`, `e2e-milestones`, and a 30-student concurrent-rating load smoke. A screenshot proves a screen rendered, not that a student was paid.

10. Threat model and known failure modes

10.1 Vote-splitting is unmitigated

The theory pairs an open field with **clustering** (grouping distinct ideas by theme, originals intact) and **synthesis** (merging genuine near-duplicates under a strict LLM equivalence check, with evaluations carried over and one-voice-per-person accounting). Agora implements neither.

The assumption is that ~30 proposals do not fragment enough to need it. The exposure is real and untested at scale: two students writing substantively the same thing genuinely do split their support, with nothing to catch it. [uncalibrated]

10.2 Declared camps assume honest positioning

The entire bridging weight depends on students placing themselves where they actually stand. Nothing detects strategic positioning (e.g. declaring an extreme to make ordinary support read as “cross-camp”). The only defence is the framing line at the positioning screen.

10.3 Coordinated rating is undetected

The anti-gaming constants bound spam and collusion in the **points economy**. They do nothing about the **score**. A class that coordinates its ratings would move consensus and bridging, and nothing currently flags it. The source text names robustness under strategic voting as an open research question for the platform generally; it is open here too.

10.4 Constitutive stages are out of scope

Absent, and by design: the “how do we decide how to decide” recursion, the stakeholder boundary, fair framing across perspectives (the question arrives pre-framed in the topic package; the multipartisan steering committee collapses into the teacher plus a generator), the recursive question/sub-question unit, subsidiarity, and incubation.

Incubation is the most substantive loss. The theory insists the genuinely creative solution — the one nobody brought to the first meeting — commonly appears at the second or third. A two-lesson arc, with lesson 1 ending at honest disagreement and lesson 2 reopening the square, may be the *intended* design rather than a fallback.

10.5 The unit is the individual, not the table

The theory’s unit is a heterogeneous table of 3–7 with mechanically enforced equal speaking time, grounded in the finding that conversational turn-taking evenness — not member IQ — predicts collective intelligence. Agora runs solo devices by default (`AgoraDeviceMode.team` exists; `TEAM_SIZE_MAX = 3`). Team-of-3 per device is closer to the theory and is an open design question.

10.6 Polarization is not actually detected

See §5.6. `SEM*` discards sign, so no scalar in the engine distinguishes unanimity from a symmetric split of equal dispersion. Inherited from the shared engine, not specific to Agora.

11. Testable predictions

Stated so they can be falsified. None has been run as a controlled study; the engine is instrumented well enough to run them.

1. **Cross-camp weighting changes what gets written.** Under `CROSS_CAMP_WEIGHT = 0.65`, proposals should reference both camps' needs at a higher rate than under an unweighted score. Measurable against the needs board vocabulary; ablatable by setting both weights to 0.5.
2. **Independent-first raises early-proposal diversity.** Pairwise embedding distance across lap-1 proposals should exceed that of a variant where the square is visible before writing.
3. **Attention allocation flattens the rating distribution.** Under `rankStalls`, the variance of ratings-per-proposal should be materially lower than under recency ordering, with no correlation between submission order and final consensus.
4. **Paying for helping raises revision rate.** Proposals receiving ≥ 1 thanked suggestion should show higher edit counts and larger `supportAtLastEdit` deltas than those receiving none. Confounded by proposal quality; needs the suggestion assignment (which `rankStalls` partly randomises) as an instrument.
5. **The FPC changes outcome classification.** Re-scoring completed sessions without the correction should move a measurable share of `success` outcomes to `honestDisagreement`, concentrated in small classes. This is a direct check on whether §5.3 is doing real work or inflating scores.
6. **Round-trip closure predicts sustained participation.** Students whose first suggestion closes a round trip should complete more subsequent laps than those whose first suggestion is declined or ignored — the engagement claim D5 rests on.

Appendix A — Constant reference

All in `packages/shared-types/src/models/agora/agoraConstants.ts` unless noted.

AGORA_BRIDGING

CONSTANT	VALUE
<code>SAME_CAMP_WEIGHT</code>	0.35
<code>CROSS_CAMP_WEIGHT</code>	0.65
<code>CENTER_CAMP_WEIGHT</code>	0.5
<code>MIN_CROSS_RATERS</code>	3
<code>CREDIT_THRESHOLD_TIER_1</code>	70
<code>CREDIT_THRESHOLD</code> (tier 2)	80

AGORA_CAMP_BOUNDS — `LEFT_MAX` 40 · `RIGHT_MIN` 60

AGORA_CYCLE — `ROUNDS` 5 · `RATINGS_PER_ROUND` 3 · `DELIBERATION_TOTAL_MS` 20 min

AGORA_SESSION — `JOIN_CODE_LENGTH` 5 (digits only) · `JOIN_CODE_UNIQUE_WINDOW_MS` 24 h ·

`DEFAULT_LESSON_MS` 45 min · `DEFAULT_ROUND_MS` 8 min · `SUCCESS_THRESHOLD` 70 · `TEAM_SIZE_MIN/MAX` 1

AGORA_POINTS — `PROPOSAL_SUBMITTED` 3 · `RATING_CREDIT` 0.5 (`RATING_CREDIT_MAX_RATINGS` 15) · `SUGGESTION_THANKED` 1 · `SUGGESTION_DECLINED` 0 · `WEAVE_CREDIT_PER_HELPER` 1 (`MAX_WEAVE_CREDITS_PER_PROPOSAL` 3) · `REVISION_CREDIT` 1 (`MAX_REVISION_CREDITS` 3) · `BRIDGING_BONUS_TIER_1` 5 · `BRIDGING_BONUS_TIER_2` 10 · `BRIDGING_BONUS` 15

AGORA_ANTI_GAMING — `MAX_OPEN_SUGGESTIONS_PER_HELPER` 2 · `MAX_WOVEN_AWARDS_PER_HELPER_PER_PROPOSAL` 2 · `MIN_REVISION_DELTA_WORDS` 3 · `REVISION_DEBOUNCE_MS` 3 min · `EDIT_HISTORY_MAX` 10

AGORA_AI_REVIEW — `RATERS_PER_CHARACTER` 3 · `MAX_ASKS_PER_CHARACTER_PER_ROUND` 5 · `LEFT_CAMP_POSITION` 10 · `RIGHT_CAMP_POSITION` 90 · `AI_UID_PREFIX` `agora-ai--`

AGORA_OUTCOME — `MIN_CROSS_RATED_PROPOSALS` 2 · `MIN_RATER_COVERAGE` 0.5

AGORA_VOTING — `DEFAULT_TOP_X` 3 · `MIN_TOP_X` 2 · `MAX_TOP_X` 10 · `DEFAULT_CUTOFF_CP` 0.5

AGORA_LIMITS — `MIN/MAX_PROPOSAL_LENGTH` 10 / 1500 · `MIN/MAX_ANSWER_LENGTH` 10 / 1000 · `RATING_BATCH_SIZE` 5

Shared consensus (`consensusCalculation.ts`) — `BAYESIAN_PRIOR_K` 2 · `CONFIDENCE_ALPHA` 0.05 · `Z_ALPHA_005` 1.645 · `AGORA_RATING_LEVELS` `[-1, -0.5, 0, 0.5, 1]` (`agoraScore.ts`)

Appendix B — Correspondence to *On Deliberation*

THEORETICAL ELEMENT	CHAPTER	STATUS IN AGORA
Grounding: shared picture sufficient to forecast	3	Implemented — period explainer, scored by the plausibility rubric at results
Bridging network scoped no larger than the issue	3	Implemented — the needs board is the shared map, persistent and one tap away
Theory of mind: state the others’ needs back until they agree	4, 9	Partial — needs board built; the interactive “you have understood me” check is scripted, not built
Heterogeneous composition as the defence against blind spots	4, 9	Implemented — declared camps, cross-camp weighted $\sim 2\times$
Anonymity against status and authority effects	9	Implemented — proposals by number; ratings never identified to owners
Independent-first against cascades	4, 9	Implemented — enforced by the cycle state machine
Anchoring countered by showing uncertainty, not point estimates	4, 9	Not implemented
Graded judgment over the blunt vote	5, 7	Implemented — 5 levels
Consensus as a conservative lower bound; confidence bought with evaluations	7	Implemented — <code>C_p</code> with $k = 2$ phantom neutral priors
Strong opposition penalised, not averaged away	6, 7	Implemented — variance term drags <code>C_p</code> down
Fair attention allocation against timing advantage	7	Partial — 3 of the source’s ~ 6 priorities; no randomness, no retirement
“What do we think” kept apart from “how sure are we”	7	Implemented — <code>mean</code> , <code>consensus</code> , <code>coverage</code> , <code>polarization</code> reported separately
Clustering and synthesis without erasing authors	7	Absent — see §10.1
Seeing the shape of an opinion (bridging proposals across clusters)	7	Implemented differently — camps declared rather than inferred
Drafting the common ground (Habermas-machine style)	7	Absent — and forbidden by D8
Telling motion from progress (paired agreement + confidence)	7	Absent as a live measure; both quantities exist per proposal
Expanding agreement at $\sim 80\%$ net support	7, 9	Diverges — composite ≥ 70 , see §8.4

THEORETICAL ELEMENT	CHAPTER	STATUS IN AGORA
Honest disagreement as a real outcome	7, 9	Implemented and extended — its own ending, gated on evidence of deliberation
Subsidiarity: devolve what need not be decided in common	8	Absent — one shared question by construction
Münchhausen recursion; stakeholder boundary; fair framing	9	Absent — the game supplies the provisional foothold
Position → need translation	9	Implemented — the needs stage and board
Criticism as service; procedural justice; minority influence	9	Implemented and extended — priced, celebrated, and named (§6)
The recursive unit (questions breeding sub-questions)	9	Absent — no floor to reach in 45 minutes
Incubation between sessions	9	Absent — see §10.4
Real-time LLM rephrasing of hostile messages	9	Deliberately refused — see D8, §7.3
From decision to execution; ownership by the proposer	10	Absent — the session ends at the verdict

Appendix C — Further reading in this repository

- `apps/agora/docs/game-script.md` — the pedagogical script: what each beat teaches, with the theoretical citation, plus the standing list of open design questions.
- `apps/agora/docs/feedback-cycle.md` — the improvement loop end-to-end: every state, notification, point, and surface, plus the decision record of what was rejected and on what grounds.
- `apps/agora/docs/chat-guide-rules.md` — **archived implementation, live policy**. The conversation invariants (I1–I9), including the “AI never writes for students” rule, outlived the code they were written for.
- `apps/agora/docs/opinion-distance-and-map.md` — the opinion-distance engine and MDS map used by the sibling Odyssey app.
- `apps/agora/CLAUDE.md` , `apps/agora/docs/HANDOFF.md` — working rules and operational gotchas.

Document generated 2026-08-19 against branch `dev`. Constants and behaviour verified against source at that commit; re-check `agoraConstants.ts` before citing numeric values.